ARCHITECTURE · INFRASTRUCTURE PLAN

JobAgent Architecture and AWS Deployment Plan

How the application is built today, the AWS setup recommended for its Python backend, and the step-by-step plan, costs, security and scaling path to get there.

Date	17 October 2026
Codebase	github.com/withhumanrevenge-cyber/JobAgents, branch `version2` at commit `8c756cc`
Product	getjobagents.xyz: AI job matching, Smart Apply, Proof of Fit, recruiter hiring
AWS region	ap-south-1 (Mumbai), closest to the Indian user base
Status	Proposal. Nothing in this document has been provisioned yet.

Contents

1. Summary and recommendation
2. Current architecture
3. Data model and security model
4. Key request flows
5. Target AWS architecture
6. Decisions and trade-offs
7. Security on AWS
8. CI/CD pipeline
9. Observability
10. Deployment plan and cutover
11. Cost estimate
12. Scaling roadmap
13. Risks and open questions
14. Appendix: configuration

1. Summary and recommendation

JobAgent has two runtime parts: a **Next.js 16 frontend** on Vercel, and a **FastAPI backend** (`backend/`) that holds all business logic: AI matching, Smart Apply, Proof of Fit, billing webhooks, email and a daily background sync. Data, sign-in and file storage live in **Supabase** (managed Postgres, Auth and Storage).

The backend is already packaged as a Docker image and is stateless apart from a few in-memory guards, which makes it a straightforward fit for **Amazon ECS on Fargate**.

> **Recommendation.** Move the backend and its daily job to AWS (ECS Fargate behind an Application Load Balancer, scheduled by EventBridge). **Keep the frontend on Vercel and the database on Supabase** for now. Moving those adds weeks of work and risk for no user-visible gain at the current scale. Revisit them with the scaling triggers in section 12.

3–4 days
One engineer: build the infrastructure, cut over, harden

≈ $55–70
Per month on AWS at launch scale (estimate, section 11)

1 task
Single backend container by design; scale up before scaling out

COMPONENT	TODAY	TARGET	DECISION
Backend API	Render web service (planned)	ECS Fargate service behind ALB	Move to AWS
Daily sync	Render cron (planned)	EventBridge Scheduler → ECS task	Move to AWS
Container images	Built by Render	Amazon ECR, built by GitHub Actions	Move to AWS
Secrets	Render env group	SSM Parameter Store (SecureString)	Move to AWS
DNS for the API	Registrar / Vercel	Route 53 record `api.getjobagents.xyz`	Move to AWS
Frontend	Vercel	Vercel	Keep
Database, Auth, Storage	Supabase	Supabase (same region as AWS if possible)	Keep
Email, AI, payments	Resend, Groq + BYOK, Razorpay / Lemon Squeezy	Unchanged	Keep
Error tracking	Sentry (backend)	Sentry + CloudWatch alarms	Keep + add

2. Current architecture

The browser talks to three places: Vercel for pages, the FastAPI backend for every `/api/...` call (authenticated with the user's Supabase access token), and Supabase directly for sign-in and a few owner-scoped reads protected by row-level security. The only server route left in Next.js is the Google OAuth callback, because it must set session cookies.

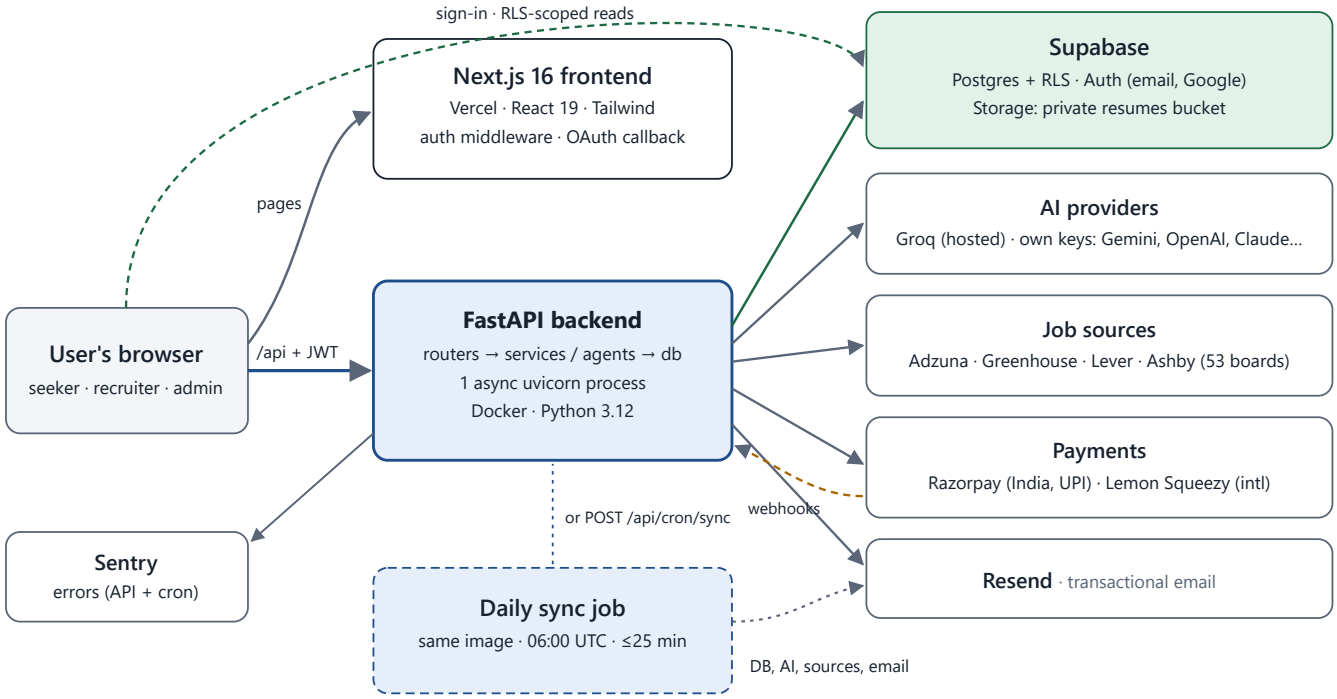


Figure 1. System context today. The backend is the only component holding service-role database credentials.

Backend modules (`backend/app/`)

LAYER	MODULES	RESPONSIBILITY
routers/	jobs, applications, resume, billing, hire, proofs, admin, ai_settings, misc	HTTP endpoints, auth and rate-limit dependencies, input validation (Pydantic). Thin.
services/	ai, llm, llm_anthropic, usage, billing, email, crypto, matches	AI mode selection (BYOK / hosted / trial), provider calls with Groq fallback, credit ledger, webhook handling, email, Fernet encryption of BYOK keys.
agents/	job_fetcher, ats_scraper, matching, writing, candidates, proof	Job ingestion and dedupe, relevance pre-ranking and AI scoring, resume/cover-letter/interview generation, candidate ranking, Proof of Fit questions and grading.
jobs/	daily_sync, backfill_descriptions	Scheduled sync (proof deadlines, pool refresh, matching, digests, follow-ups); one-off data repair.
core	main, config, auth, db, errors, ratelimit, plans, observability	App wiring, settings from env, Supabase token check, service-role client, error shape, in-memory rate limits, plan limits shared with the frontend, Sentry.

3. Data model and security model

The schema is `supabase/schema.sql` plus migrations `002`–`013`, all idempotent. Every table has row-level security; the backend uses the service-role key, which bypasses it, so every backend query filters by the authenticated user's id explicitly.

TABLE	HOLDS	BROWSER ACCESS
profiles	One per user: contact, parsed resume, targets, plan, account type, talent-pool opt-in	Own row (billing columns guarded by trigger)
jobs	Shared, deduplicated job pool from all sources	Read only
matches	Per user × job: score, reason, skills, status, tailored resume, cover letter, interview prep	Own rows
usage_events	Credit ledger and token metering (hosted vs BYOK)	None; functions callable by service role only
user_ai_keys	Users' own AI keys, Fernet-encrypted	None (deliberately no policies)
webhook_events	Idempotency for payment webhooks	None
job_postings	Recruiter roles, response window (`response_sla_days`)	Recruiter's own
candidate_matches	Candidate × posting: fit score, invitation, proof, decision, deadline, intro response	Recruiter read only (012)
skill_proofs	Proof of Fit steps, timings, report, share slug	Owner read only
storage: resumes	Uploaded resume PDFs, private bucket	Owner's folder only

Security controls already in the code

- **Auth:** every API call carries the Supabase access token; the backend verifies it with Supabase Auth (30-second cache).
- **Admin:** requires both an allow-listed email and `profiles.is_admin`.
- **Webhooks:** HMAC-SHA256 signatures, constant-time comparison, idempotent by event id.
- **Cron:** `/api/cron/sync` refuses every call unless `CRON_SECRET` matches.
- **BYOK keys:** encrypted at rest with rotating Fernet keys; never returned to the browser; fixed provider base URLs.
- **CORS:** only the app origin (plus configured previews); no cookies.
- **Privacy:** recruiters get contact details only after the candidate accepts an introduction.
- **Errors:** internal details stay in logs; Sentry sends no request bodies or IPs.

> **Constraint that shapes the AWS design:** rate limits, the per-user match lock, the per-proof answer lock and the "sync already running" guard live in process memory. They are correct for **one** backend process. The AWS service therefore runs one task; section 12 lists what changes before running more.

4. Key request flows

Find jobs

1. `POST /api/jobs/fetch` pulls Adzuna and company boards for the user's roles and country into the shared pool.
2. The browser loops `POST /api/match`; each call ranks unscored jobs by keyword overlap, then AI-scores up to 24 (4 at a time).
3. Scores are saved to `matches`; the loop ends when `remaining` is 0 or the plan cap is hit.

Smart Apply

1. Consent modal, then `POST /api/apply/smart`.
2. Credits are reserved atomically (`consume_credits`) unless the user's own key is used.
3. Two quality-model calls: tailored resume (with ATS score) and cover letter. Credits are refunded if either fails.

Proof of Fit

1. `POST /api/proofs` writes 3 role-specific questions (our key, monthly cap).
2. Each answer is timed by the server; a follow-up is generated from it.
3. After 6 answers, grading scores skills with verified quotes; an invited proof starts the recruiter's decision clock.

Payments and daily sync

1. Checkout is created by the API; Razorpay or Lemon Squeezy calls the signed webhook, which updates the plan once per event.
2. Daily at 06:00 UTC: expire overdue recruiter decisions, refresh the job pool, match active users, send digests and 7-day follow-ups.

Timing note for the load balancer: the longest synchronous calls (Smart Apply, proof grading) take tens of seconds, so the ALB idle timeout is raised to 120 s (section 5).

5. Target AWS architecture

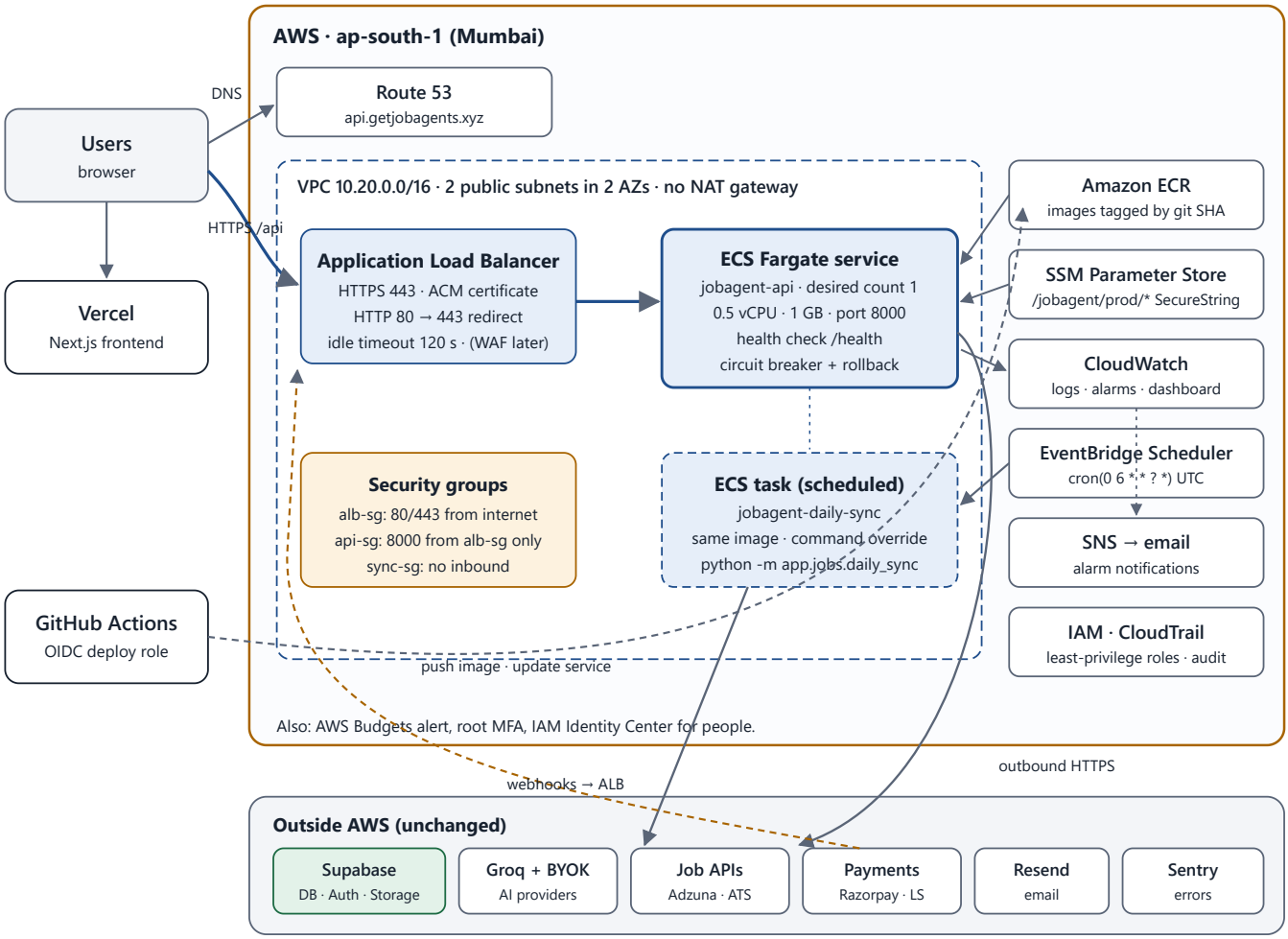


Figure 2. Target deployment. The API task has a public IP for outbound calls but accepts traffic only from the load balancer's security group.

RESOURCE	CONFIGURATION
VPC	`10.20.0.0/16`, two public subnets (`ap-south-1a`, `ap-south-1b`), internet gateway, no NAT gateway.
ALB	Internet-facing, listeners 443 (TLS 1.2+, ACM cert for `api.getjobagents.xyz`) and 80 → 443 redirect. Target group: IP targets, port 8000, health check `/health` every 15 s, 2 healthy / 3 unhealthy. Idle timeout 120 s. Deregistration delay 30 s.
ECS cluster	`jobagent-prod`, Fargate capacity provider, Container Insights on.
API service	Task definition `jobagent-api`: 0.5 vCPU, 1 GB, x86_64 Linux, image from ECR by git SHA, port 8000, secrets from SSM, logs to `/ecs/jobagent-api` (30-day retention). Desired count 1; deployment min 100% / max 200%; circuit breaker with rollback; health-check grace 60 s.
Daily sync	Task definition `jobagent-daily-sync`: same image, command `python -m app.jobs.daily_sync`, 0.5 vCPU / 1 GB, logs to `/ecs/jobagent-daily-sync`. EventBridge Scheduler, `cron(0 6 * * ? *)` UTC, retry policy 0 (the job is idempotent but long).
ECR	Repository `jobagent-api`, immutable tags, scan on push, lifecycle: keep last 20 images.
Secrets	SSM Parameter Store SecureString under `/jobagent/prod/` (appendix), default KMS key.
DNS	Route 53 hosted zone for the domain (or a delegated `api.` subdomain); alias A/AAAA record to the ALB.

6. Decisions and trade-offs

DECISION	WHY	COST OF THE CHOICE
ECS Fargate	Runs the existing Dockerfile unchanged; no servers to patch; native scheduled tasks, rolling deploys with automatic rollback, IAM roles per task.	More setup than a PaaS; ALB is a fixed monthly cost.
Not EKS	Kubernetes adds a ~$73/month control plane and real operational load for one container.	Revisit only with many services.
Not a single EC2 + Docker	Cheapest (~$15/month), but you own OS patching, TLS renewal, restarts and deploy scripts.	Valid budget option; listed in section 11.
Public subnets, no NAT	A NAT gateway costs ~$35+/month plus data. The task gets a public IP for outbound calls; its security group admits only the ALB.	Task is internet-addressable but closed; move to private subnets + NAT when compliance or scale requires.
One API task	In-memory rate limits and locks are correct only in one process (section 3). Fargate restarts a failed task automatically.	A task restart is a ~1 minute outage; deploys briefly run two tasks, which is harmless.
SSM Parameter Store	~30 values; standard parameters are free, Secrets Manager costs $0.40 per secret per month and adds rotation the app doesn't need.	No automatic rotation (rotate by hand, then redeploy).
Keep Supabase	Auth, RLS policies, storage and the browser's direct reads are all built on it. Replacing it means Cognito + RDS + S3 + rewriting auth on both sides.	Database is outside AWS; keep both in the same region to limit latency.
Keep Vercel	Best-supported host for Next.js 16 (middleware, server components, previews). Amplify Hosting is possible later.	Two vendors for one app.
ap-south-1	Lowest latency to Indian users and closest to Indian payment and email traffic.	Slightly higher unit prices than us-east-1.

7. Security on AWS

Accounts and people

- Root user: hardware or app MFA, no access keys, used only for billing.
- People sign in through IAM Identity Center with MFA; no long-lived IAM user keys.
- CloudTrail on in all regions; AWS Budgets alert at 80% and 100% of the monthly budget.
- Optional: separate `staging` and `prod` accounts under AWS Organizations.

Network

- `alb-sg`: inbound 80/443 from anywhere.
- `api-sg`: inbound 8000 only from `alb-sg`. Required because uvicorn trusts forwarded headers (`--forwarded-allow-ips='*'`).

IAM roles (least privilege)

- **Task execution role:** pull from ECR, write its log groups, read `/jobagent/prod/*` parameters and decrypt with their KMS key.
- **Task role:** none needed today (the app calls no AWS APIs). Keep it empty.
- **Scheduler role:** `ecs:RunTask` on the sync task definition and `iam:PassRole` for its two roles.
- **GitHub deploy role:** assumed through OIDC, restricted to this repository's `main` branch; push to the one ECR repository, register task definitions, update the one service.

Data in transit

- TLS at the ALB; all outbound calls (Supabase, AI, payments) are HTTPS.

- `sync-sg`: no inbound. Outbound 443 for both tasks.
- Phase 3: AWS WAF on the ALB with AWS managed rules and a per-IP rate rule.

8. CI/CD pipeline



Pull requests run only the checks. Frontend deploys stay on Vercel's own Git integration. Migrations are applied by hand before the deploy that needs them.

Figure 3. One GitHub Actions workflow (`.github/workflows/deploy-api.yml`), authenticating to AWS through OIDC; no AWS keys are stored in GitHub.

- **Rollback:** automatic if the new task fails health checks (deployment circuit breaker). Manual: `aws ecs update-service --task-definition jobagent-api:<previous>`, which takes about two minutes.
- **Database changes:** migrations are idempotent and additive; apply them in the Supabase SQL editor (staging first), then deploy. Never deploy code that needs a migration before the migration has run.
- **Staging:** a second service (`jobagent-api-staging`, its own SSM path and Supabase project) deployed from the `version2` /feature branches. Optional for launch, recommended before onboarding colleges.

9. Observability

SIGNAL	SOURCE	ALARM (→ SNS EMAIL)
API errors	ALB `HTTPCode_Target_5XX_Count`	> 10 in 5 minutes
API down	Target group `HealthyHostCount`	< 1 for 2 minutes
Latency	ALB `TargetResponseTime` p95	> 30 s for 10 minutes (AI calls are slow by nature)
Memory / CPU	ECS service metrics	Memory > 85% or CPU > 80% for 15 minutes
Daily sync failed	EventBridge / ECS task exit code; log metric filter on "Daily sync crashed" and "Proof deadlines failed"	Any occurrence
Daily sync missing	Log metric filter on "Daily sync done"	Missing for 26 hours
Application exceptions	Sentry (`SENTRY_DSN`)	Sentry's own alert rules
Spend	AWS Budgets	80% and 100% of monthly budget

Product metrics (signup → interview funnel, Proof of Fit outcomes) are on the in-app admin page, backed by `admin_funnel()` (migration 013).

10. Deployment plan and cutover

Infrastructure should be written as code (Terraform or AWS CDK) and kept in the repository under `infra/`, so production can be rebuilt and reviewed. The steps below are the order of work either way.

PHASE 0 · HALF A DAY · PREREQUISITES

Account and inputs

- ☐ Create the AWS account; root MFA; IAM Identity Center admin user; enable CloudTrail; Budgets alert (e.g. $100).
- ☐ Confirm the Supabase project's region; if it isn't Mumbai, note the added latency (section 13).
- ☐ Decide DNS: move the zone to Route 53, or delegate only `api.getjobagents.xyz`.
- ☐ Collect every backend secret (appendix) from the current `.env` / dashboards.

PHASE 1 · 1–1.5 DAYS · BUILD

Infrastructure as code

- ☐ VPC, subnets, internet gateway, security groups.
- ☐ ECR repository; push a first image by hand to prove the build.
- ☐ SSM parameters under `/jobagent/prod/`.
- ☐ ECS cluster, both task definitions, IAM roles, log groups.
- ☐ ACM certificate (DNS validation), ALB, target group, listeners, Route 53 alias.
- ☐ API service (desired count 1) with circuit breaker; confirm `https://api.getjobagents.xyz/health` returns `{"status":"ok"}`.
- ☐ EventBridge schedule created **disabled** (enabled at cutover so two schedulers never run together).
- ☐ GitHub OIDC provider and deploy role; `deploy-api.yml` workflow; first deploy through the pipeline.
- ☐ CloudWatch alarms and SNS email subscription (confirm the subscription email).

PHASE 2 · HALF A DAY · CUTOVER (LOW-TRAFFIC HOUR, IST NIGHT)

Switch traffic

1. Apply migrations `009`–`013` in Supabase (staging first) if not yet applied.
2. Smoke test against the AWS URL with a test account: sign in, upload resume, Find jobs, Smart Apply, start and finish a Proof of Fit, recruiter invite and decision, admin page.
3. Vercel: set `NEXT_PUBLIC_API_URL=https://api.getjobagents.xyz`; set `CORS_ORIGINS` in SSM for preview URLs if used; redeploy the frontend.
4. Razorpay and Lemon Squeezy dashboards: webhook URLs to `https://api.getjobagents.xyz/api/webhooks/razorpay` and `/lemonsqueezy`; send a test event from each.
5. Disable any other scheduler (Render cron, GitHub cron), then enable the EventBridge schedule. Run the sync task once by hand and read its log.
6. Watch alarms, Sentry and logs for 24 hours. Keep the previous host running but idle for one week, then delete it.

PHASE 3 · 1 DAY · HARDENING

After launch

- ☐ AWS WAF on the ALB: AWS managed common rule set + per-IP rate limit.
- ☐ Staging environment and a written runbook (deploy, rollback, rotate a secret, restore Supabase backup).
- ☐ Supabase: confirm daily backups / point-in-time recovery on the paid plan; test a restore into staging.
- ☐ Rotate any secret that was ever pasted into chat, email or a shared document.

Rollback at cutover: repoint `NEXT_PUBLIC_API_URL` and the webhooks back to the previous host and redeploy Vercel. Both hosts use the same Supabase database, so no data moves in either direction.

11. Cost estimate

Approximate on-demand prices for ap-south-1 at launch scale (one API task running 24/7, sync job ~30 minutes a day, light traffic). **These are estimates; confirm with the AWS Pricing Calculator before committing.** AI, Supabase, Vercel, Resend and payment fees are unchanged and not included.

ITEM	BASIS	PER MONTH (USD)
Fargate: API task	0.5 vCPU + 1 GB × 730 h	≈ 18–22
Application Load Balancer	Hourly charge + low LCU usage	≈ 18–25
Public IPv4 addresses	ALB in 2 AZs + API task, $0.005 per IP-hour	≈ 8–11
Fargate: daily sync	0.5 vCPU + 1 GB × ~15 h	≈ 0.5
CloudWatch	Logs (a few GB), ~8 alarms, Container Insights	≈ 3–8
Route 53, ECR, SSM, SNS, Scheduler	One zone, <2 GB images, standard parameters	≈ 1–2
Total (Phase 1–2)		**≈ 55–70**
+ AWS WAF (Phase 3)	Web ACL + a few rules + requests	≈ 8–12
+ Staging environment	Second small service, shared ALB via host rule	≈ 12–20

Cheaper alternatives, if cost matters more than operations

OPTION	APPROX. PER MONTH	TRADE-OFF
Single EC2 (t3.small) + Docker + Caddy	≈ 15–20	You patch the OS, handle restarts, write deploy scripts; cron via the host.
Lightsail container service	≈ 10–25	Simple, includes HTTPS endpoint; no scheduled tasks (use the `/api/cron/sync` endpoint with an external scheduler), fewer controls.
Render (already configured)	≈ 14	Least work; not AWS; fewer network and IAM controls.
Fargate on Graviton (ARM)	≈ −20% compute	Build a `linux/arm64` image; all dependencies have ARM wheels, but test first.

Startups can apply for AWS Activate credits, which typically cover the first months of a setup this size.

12. Scaling roadmap

SIGNAL	ACTION
CPU or memory alarms, or p95 latency rising with traffic	Scale **up** the one task: 1 vCPU / 2 GB, then 2 vCPU / 4 GB. No code changes.
Need more than one API task (availability or load)	Move rate limits and the per-user / per-proof locks to a shared store (ElastiCache Serverless for Redis/Valkey, or Postgres advisory locks); then set desired count 2+ with target-tracking autoscaling.
Daily sync exceeds its 25-minute budget	Split by user batches into several scheduled tasks, or move matching to an SQS queue with a worker service.
Long AI calls hold connections (Smart Apply, grading)	Make them asynchronous: enqueue to SQS, worker writes result, browser polls (the matching loop already works this way).

SIGNAL	ACTION
Groq rate limits at placement-season peaks	Set the hosted fallback provider; raise `HOSTED_MAX_CONCURRENCY` only with a higher Groq tier.
Compliance or enterprise/college contracts need data in AWS	Private subnets + NAT; consider RDS for Postgres with logical replication from Supabase as a first step; keep Supabase Auth until a Cognito migration is planned.

13. Risks and open questions

RISK	IMPACT	MITIGATION
Supabase region differs from Mumbai	Every API call adds cross-region round trips (tens to 200+ ms each; several per request).	Check first. If needed, run AWS in the Supabase region instead, or plan a Supabase region move.
Single task restart	~1 minute of API errors while Fargate replaces it.	Health checks + alarms; scale-out path in section 12 when this matters.
Two schedulers at once	Duplicate digests and follow-up emails.	Schedule created disabled; enable only after the old cron is off.
Secrets exposure	Service-role key gives full database access.	SSM SecureString, least-privilege execution role, no secrets in images or GitHub; rotate after any leak.
Migration/code order	Errors if code ships before its migration.	Migrations first, staging first; all migrations are idempotent.
Cost drift	Logs, NAT or WAF added later raise the bill.	Budgets alerts; log retention 30 days; review monthly.
Untested in production	Proof of Fit and the AWS stack have only passed automated checks.	Full smoke test at cutover; keep the old host for a week.

Appendix: configuration

Each backend setting becomes an SSM parameter under `/jobagent/prod/` and is mapped into both task definitions as an environment variable of the same name. Non-secret values may be plain `environment` entries instead.

GROUP	VARIABLES	TYPE
App	`ENVIRONMENT=production`, `APP_URL=https://getjobagents.xyz`, `CORS_ORIGINS`, `ADMIN_EMAILS`	Plain
Database	`SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`	Secret
AI	`GROQ_API_KEY`, `GROQ_MODEL`, `GROQ_FAST_MODEL`, `HOSTED_MAX_CONCURRENCY`, `HOSTED_FALLBACK_PROVIDER`, `HOSTED_FALLBACK_API_KEY`, `HOSTED_FALLBACK_FAST_MODEL`, `HOSTED_FALLBACK_MODEL`, `BYOK_ENCRYPTION_KEY`	Keys secret
Jobs	`ADZUNA_APP_ID`, `ADZUNA_APP_KEY`	Secret
Email	`RESEND_API_KEY`, `RESEND_FROM`	Key secret
Cron	`CRON_SECRET`	Secret
Razorpay	`RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, `RAZORPAY_WEBHOOK_SECRET`, `RAZORPAY_PRO_PLAN_ID`, `RAZORPAY_PREMIUM_PLAN_ID`	Secrets / IDs
Lemon Squeezy	`LEMONSQUEEZY_API_KEY`, `LEMONSQUEEZY_STORE_ID`, `LEMONSQUEEZY_PRO_VARIANT_ID`, `LEMONSQUEEZY_PREMIUM_VARIANT_ID`, `LEMONSQUEEZY_WEBHOOK_SECRET`	Secrets / IDs
Monitoring	`SENTRY_DSN`	Secret
Frontend (Vercel)	`NEXT_PUBLIC_API_URL`, `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`	Public

Never rotate `BYOK_ENCRYPTION_KEY` by replacing it: add the new key first in the comma-separated list (newest first), deploy, and remove the old one only after every saved key has been re-encrypted; otherwise users' saved AI keys become unreadable.

Resource names used in this document: cluster `jobagent-prod`; services/task definitions `jobagent-api`, `jobagent-daily-sync`; ECR `jobagent-api`; log groups `/ecs/jobagent-api`, `/ecs/jobagent-daily-sync`; parameters `/jobagent/prod/<NAME>`.